

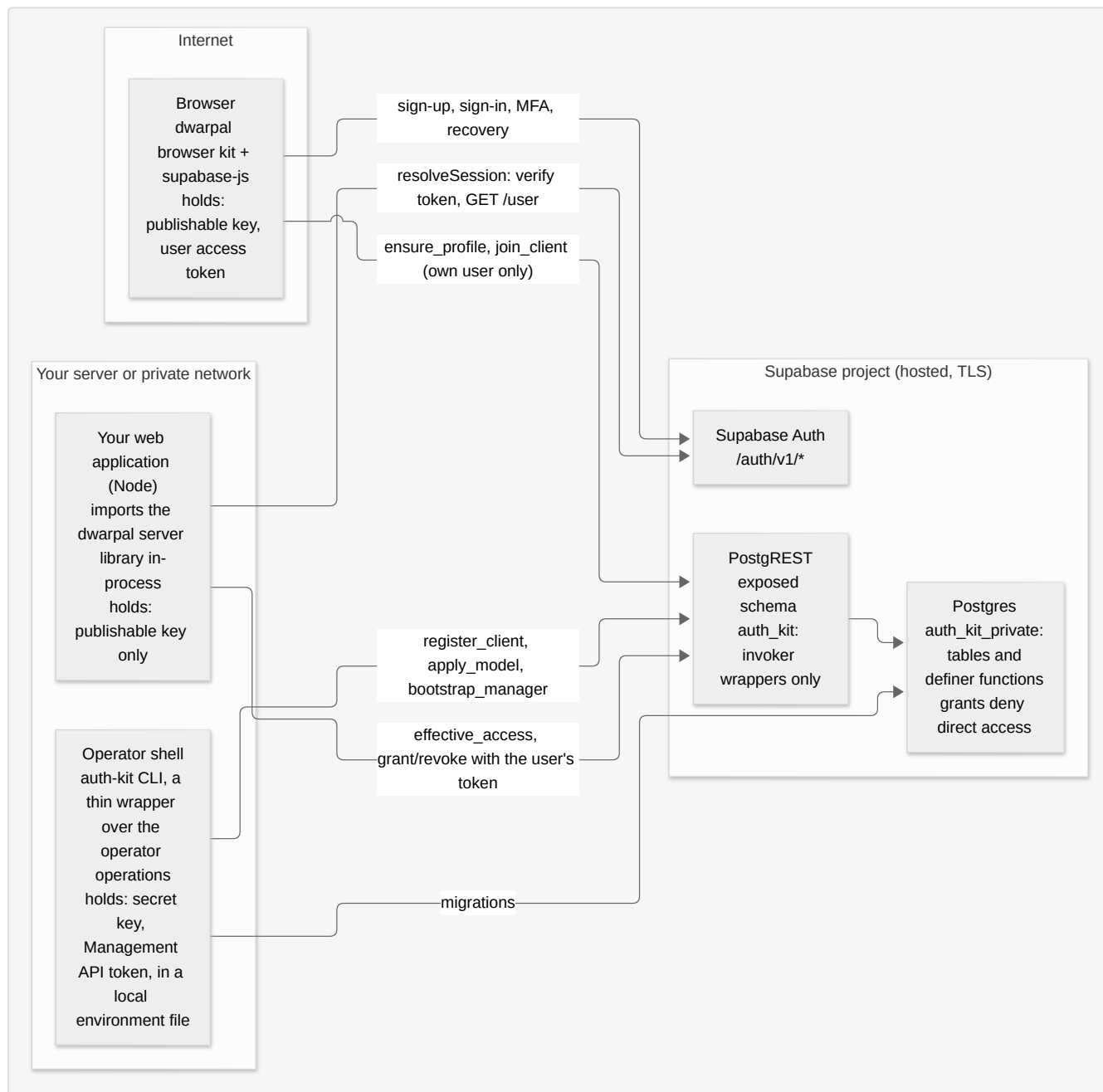
Dwarpal RBAC low-level design

This document explains how Dwarpal's roles, permissions and memberships work in SQL, why the design is shaped this way, and how it behaves under retries, concurrency and partial failure. The contract itself is in the [design](#); integration steps are in the [user manual](#).

Diagrams are Mermaid and render on GitHub.

1. Three surfaces, one contract

Dwarpal is a **library plus SQL**, not a service. It has no network API of its own, so there is nothing that browsers or external services can call beyond Supabase's own endpoints and a small set of SQL wrappers. Your web application uses the library under a fixed contract; a client configures only runtime data and its own configuration.



Surface	Caller	Credential	Can do
Browser	the end user's browser	publishable key and that user's access token	Supabase Auth endpoints; two wrappers that act only on the caller's own user
Server library	your web application, in-process	publishable key and the request's access token	resolve a session, compute permissions, forward manager actions under the manager's own token
Operator	a person or deploy step with the project's secret key	secret key and Management API token	migrations, client registration, permission model, first manager

What a client **cannot change**: function signatures, the SQL functions, their grants and the invariants in section 5. These ship in the package and its versioned migrations.

What a client **configures**: `ClientConfig` (branding, routes, providers, session policy) and the **model data** (roles, permissions, role-to-permission map, memberships), all written through the kit's own functions.

Two limits follow from the platform:

- **The data is hosted.** Supabase runs in Supabase's cloud, so "same machine" applies to your web application and the library, not to the database. Every hop to Supabase is TLS with a per-user token or the operator key.
- **The secret key is root for the project.** Supabase's `service_role` bypasses row security. A secret-key holder who deliberately writes tables instead of calling the functions can break the invariants. The kit keeps the secret key off the web application and the browser, records every sanctioned write as an event, and `auth-kit doctor` reports drift such as memberships without a matching event. This is inherent to Supabase.

2. Why RBAC, and where it stops

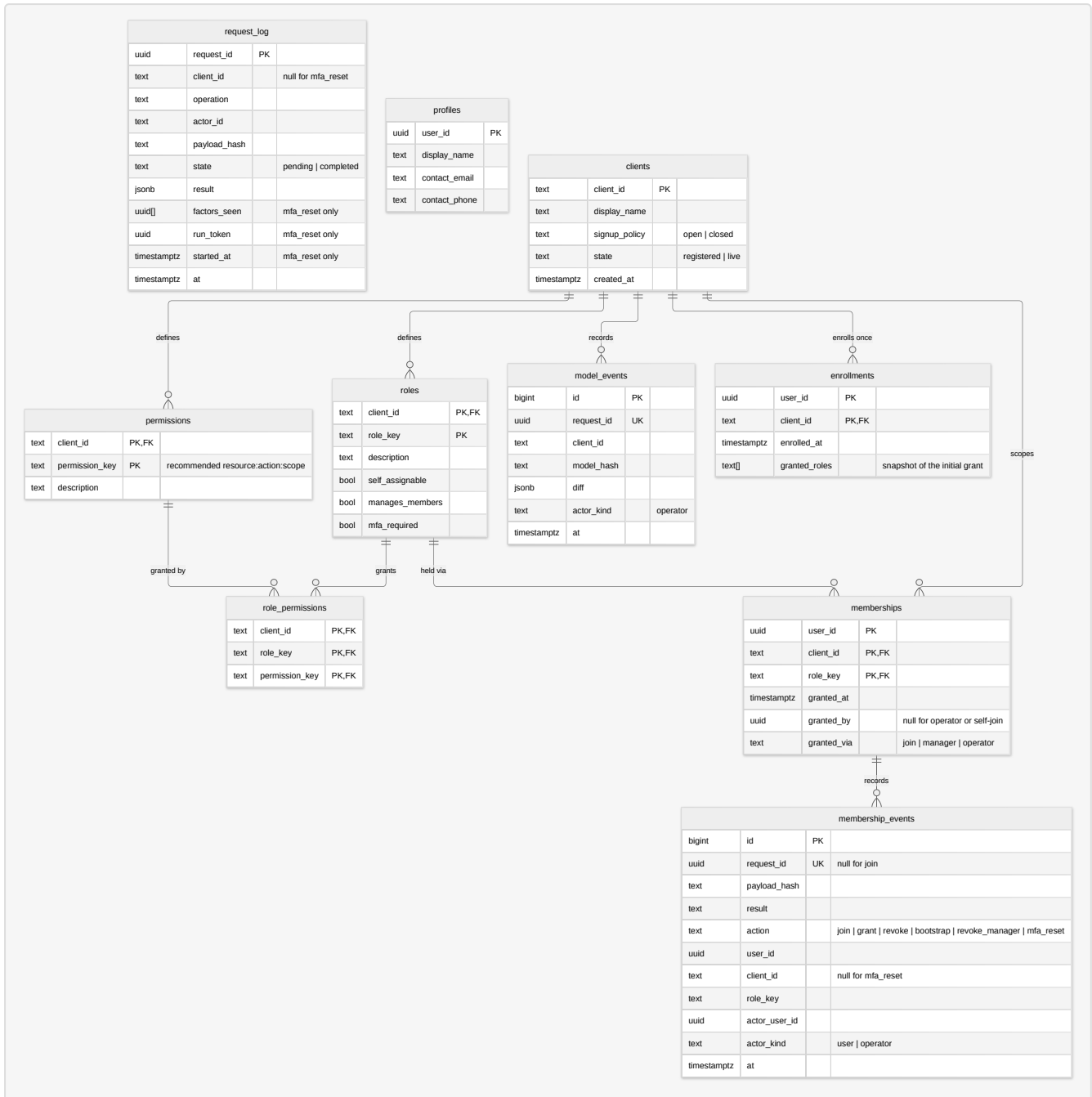
The simplest alternative is a fixed set of role names, with each consumer deciding in code what a role may do, for example `requireRole(principal, ['staff', 'admin'])`. That is smaller and typo-proof at compile time, but it does not generalise:

Aspect	Fixed role names	Client-defined roles with RBAC
Adding a role	a code change in the kit and every consumer	a data change
What a role may do	scattered across consumers, which can disagree	one model per client, one answer
Consumer check	role-name comparison	permission-key check
A typo	compile error	denied at runtime (fail closed); <code>explain()</code> shows why
Reusable for another website	no	yes
Row-level ownership	consumer	consumer

The limits of the RBAC design are deliberate:

1. **RBAC is type-level, not row-level.** It answers "may a buyer read orders", never "may this buyer read order 123". The consumer keeps its ownership check. Going further needs a relationship engine such as OpenFGA or SpiceDB, which needs a server this stack does not have.
2. **A wrong model is a data bug.** A role mapped to the wrong permission is silent until tested. Mitigations: `apply_model` validates the whole model before writing, `doctor` reports unmapped permissions and roles with no members, and the hosted actor matrix runs per model state.
3. **Several roles per user need union semantics with an MFA subtlety.** Section 4.3 defines them exactly.

3. Data model



Constraints that carry design intent:

- roles**: check (not (self_assignable and manages_members)). A role that public sign-up can obtain can never manage members.
- role_permissions**: composite foreign keys on (client_id, role_key) and (client_id, permission_key), so a role cannot be mapped to another client's permission.
- memberships** → **roles**: on delete restrict. A role in use cannot be deleted; revoke first.
- role_permissions** → **permissions**: on delete restrict. A mapped permission cannot be deleted; unmap first. `apply_model` orders its own operations so that a single model-file change still applies in one transaction.
- One user may hold several roles for one client. Effective permissions are the union (section 4.3).
- enrollments**: primary key (user_id, client_id), written once by a successful `join_client`, never updated or deleted by any function. It is the durable fact "this user's public initial grant happened",

independent of which memberships still exist. A manager's revoke therefore sticks: the next sign-in finds the row and grants nothing.

- `clients.state`: `registered` on `register_client`, `live` on the first successful `bootstrap_manager`. Once live, `apply_model` refuses a model that would leave the client with no assigned manager.
- `request_log`: one row per request-bearing command, written **whether or not anything changed**. The retry rule reads this table, not the event tables: same id and hash → stored `result`; same id and another hash → `request_conflict`. Events record mutations; `request_log` records outcomes. The fingerprint is `sha256(canonical JSON of {operation, client_id, actor_id, payload})`, so an id is bound to its actor and cannot be replayed by another manager. `state`, `factors_seen`, `run_token` and `started_at` exist for `mfa_reset` only: `run_token` names the current claim holder and `started_at` is reset on takeover. Every other command inserts a `completed` row in its own transaction.
- `membership_events.payload_hash` and `result` are kept for audit display; they are not the retry lookup.

All tables except `profiles` live in `auth_kit_private`. No PostgREST role has direct table privileges. The exposed schema `auth_kit` holds only invoker wrappers, the RLS helpers, `profiles` and the `public_clients` view. There are two different fences:

- PostgREST **routing** reaches only schemas in the project's exposed list, and `auth_kit_private` is never listed, so no HTTP call can address an implementation.
- SQL **privileges** (`EXECUTE`, `USAGE`) stop a caller who already runs SQL as `authenticated`.

Both are asserted, and they are tested differently: routing over HTTP, privileges by SQL impersonation.

4. Reserved flags, functions and evaluation

4.1 The three reserved flags

Flag	Meaning the kit enforces	Set by
<code>self_assignable</code>	<code>join_client</code> grants this role to a verified user at their one enrollment	operator, in the model
<code>manages_members</code>	a holder may grant and revoke non-manager roles for this client; bootstrap must name such a role	operator, in the model
<code>mfa_required</code>	permissions from this role count only in an <code>aal2</code> session; the browser kit prompts enrolment	operator, in the model

Everything else about a role is client data: its key, description and permission set.

4.2 Functions and who may call them

Each function is a wrapper in `auth_kit` that calls an implementation in `auth_kit_private`. The full rules are in [design section 4.6](#); this table summarises the checks.

Function	Caller	Checks inside SQL
<code>ensure_profile()</code>	authenticated user	upserts own row only
<code>join_client(client_id)</code>	authenticated user	lock; email confirmed (from <code>auth.users</code> , not the token); <code>unknown_client</code> if missing; <code>already_enrolled</code> if an enrollment row exists, writing nothing whatever the current memberships are; closed if <code>signup_policy = closed</code> ; <code>no_default_role</code> if no <code>self_assignable</code> role (nothing written, retryable); otherwise inserts the enrollment row, one membership per <code>self_assignable</code> role and one <code>join</code> event each, and returns <code>enrolled</code> . One transaction.

Function	Caller	Checks inside SQL
<code>effective_access(client_id)</code>	authenticated user	returns <code>enrolled_at</code> and own memberships with each role's flags and permissions; read-only; no lock
<code>grant_membership(user_id, client_id, role_key, request_id)</code>	authenticated manager	lock, then the <code>request_id</code> check, then manager authority: at least one held <code>manages_members</code> role for that client is active ; held but none active → <code>mfa_required</code> ; none held → <code>forbidden</code> ; target role is not <code>manages_members</code> ; target user exists and is confirmed; not self. The only path that restores a revoked self-assignable role.
<code>revoke_membership(user_id, client_id, role_key, request_id)</code>	authenticated manager	same caller checks; cannot revoke a <code>manages_members</code> role
<code>bootstrap_manager(user_id, client_id, role_key, request_id)</code>	operator (<code>service_role</code> only)	lock; role has <code>manages_members</code> ; user confirmed; idempotent; sets <code>clients.state = live</code>
<code>revoke_manager(user_id, client_id, role_key, request_id)</code>	operator	lock; <code>request_id</code> check; refuses if it would leave zero managers for the client
<code>register_client(client_id, display_name, signup_policy)</code>	operator	idempotent on <code>client_id</code> ; no <code>request_id</code>
<code>apply_model(client_id, model jsonb, request_id, dry_run)</code>	operator	lock; validates the whole model, including two live-client guards: never set <code>manages_members</code> on a role that has holders, and on a <code>live</code> client never leave zero assigned managers; computes the diff annotated with <code>reach</code> (<code>future_joiners</code> for <code>self_assignable</code> changes, <code>holders: N</code> for mapping changes); applies in one transaction; one <code>model_events</code> row
<code>export_model(client_id)</code>	operator	read-only; returns the current model in file format plus the last applied hash
<code>mfa_reset_begin</code> , <code>mfa_reset_note</code> , <code>mfa_reset_finish</code>	operator	project-wide lock; reserve-then-act with a single claim (section 4.4)

Promotion to a manager role is **operator only**, and only through `bootstrap_manager`. Neither a manager nor a model change can promote existing holders, so a compromised manager account cannot mint more managers. This is stricter than typical admin consoles.

One lock, checked-after semantics. Every write function, including `join_client`, begins with `pg_advisory_xact_lock(hashtext('auth_kit:' || client_id))` and only then checks authority and state. Model writes and membership writes for one client are therefore serialised against each other, and a manager revoked by a concurrent transaction is seen. Reads never wait.

Request semantics. Every request-bearing function writes a `request_log` row with the fingerprint and the result **on every first call, including no-ops** (unchanged model, `already_member`, `not_member`). A retry with the same id and payload returns the stored result and writes nothing. The same id with a different payload raises `request_conflict` and writes nothing. There is no third outcome. Storing the outcome only in event rows would not be enough: a no-op leaves no event, so a changed-payload retry after a no-op would otherwise succeed.

4.3 Evaluation: from the live snapshot, in-process

`resolveSession` makes one live call to Supabase Auth and one `effective_access(client_id)` read per request, which returns roles, flags and permissions in one round trip. Every check after that is a pure local function.

```

can(principal, 'orders:read:any'): boolean
canAll(principal, ['orders:read:any', 'orders:cancel:any']): boolean
canAny(principal, [...]): boolean
explain(principal, 'orders:read:any'): { allowed: boolean; via: string[]; withheld: { role:
string; reason: 'mfa_required' }[] }
requirePermission(principal, 'orders:read:own') // throws AuthError('forbidden') or
AuthError('mfa_required')

```

Union rule: a permission is held if at least one **active** role grants it. A role is active when it is not `mfa_required`, or when the session is `aal2`. A user holding `buyer` and `agent` at `aal1` keeps the buyer's permissions and sees `mfaPending: true`; `explain` names `agent` as `withheld`. `requireRole` and the SQL `has_role` test **active** roles only; `access.roles` is display data.

Own versus any. Union means a lesser role can hand a user a key, so one key must never mean both "own rows" and "all rows". Convention: `orders:read:own` and `orders:read:any`. `own` refers to the row being acted on; an action whose narrow scope is the principal as the target, such as assigning a lead to oneself, uses `:self` (`leads:assign:self`, with the guard "target assignee equals the principal's user id"). The consumer guard, in this exact order:

```

if (!can(principal, 'orders:read:any')) { // broad branch needs an active role that
grants it
  requirePermission(principal, 'orders:read:own'); // forbidden or mfa_required
  if (order.userId !== principal.identity.userId) throw new AuthError('forbidden');
}

```

The matching RLS policy for a consumer table in Supabase Postgres:

```

using (
  auth_kit.has_permission('acme', 'orders:read:any')
  or (auth_kit.has_permission('acme', 'orders:read:own') and owner_id = auth.uid())
)

```

A user with `buyer` and an inactive `agent` role fails the first test on both paths and is stopped by ownership on the second.

Two typical questions, answered by the API:

- "Does user A with Role1 have access to R4?" → `can(principal, 'R4:access')` is `false`, and `explain` shows no active role grants it.
- "Does A have access to R2, R3, R4 and R5?" → `canAll` is `false`; `explain` per key shows R2 and R3 via Role1, and R4 and R5 with no grantor.

4.4 MFA reset: reserve, then act, under one claim

Deleting a user's MFA factors happens in Supabase Auth through the admin API, while the audit lives in SQL, so the operation cannot run in one transaction. The design binds the request id in SQL before any Auth call and lets exactly one runner hold the right to complete it.

- `mfa_reset_begin` binds the id to `{mfa_reset, operator, user_id}` in a pending `request_log` row under the project-wide lock `hashtext('auth_kit:mfa_reset')`, before any Auth call. It answers `proceed` (new row, fresh `run_token`), the stored result (already completed), `resume` (a pending row older than the

120 s lease, with its recorded factor list), `request_in_progress` (pending inside the lease) or `request_conflict` (another user behind the same id).

- **Single claim.** On `resume`, the same statement under the lock replaces the row's `run_token` and sets `started_at = now()`. The next caller therefore sees a pending row inside the lease and gets `request_in_progress`. Without this, every caller after the lease would get `resume` and two takeovers could both reach Auth.
- `mfa_reset_note` records the factor list once, before the first deletion. `mfa_reset_finish` completes the row and writes the one event, with `client_id` null.
- Both check the caller's `run_token` **before** the row state and raise `run_superseded` on a mismatch without writing, so one runner completes the row and one event exists.
- The CLI stops itself with `lease_expired` before any admin API call once its claim is older than the lease minus the 20 s per-call timeout (100 s), measured on its own clock from the moment it sent `begin`. A slow runner therefore does not race a takeover. A superseded runner that reaches Auth late can only repeat deletions of already-recorded factors, which are idempotent.
- The factor list is never part of the fingerprint. A retry after success returns the stored result without touching Auth; a different user behind the same id conflicts before Auth; a crash mid-deletion resumes with the original list.

The complete outcome tables are in [design section 4.7](#).

5. Invariants the kit enforces

#	Invariant	Where enforced
I1	Public sign-up and join can yield only <code>self_assignable</code> roles	<code>join_client</code>
I2	A <code>self_assignable</code> role is never <code>manages_members</code>	table check constraint, <code>apply_model</code>
I3	Only a manager of that client, or the operator, changes memberships	<code>grant_membership</code> , <code>revoke_membership</code>
I4	Managers cannot grant or revoke manager roles	<code>grant_membership</code> , <code>revoke_membership</code>
I5	A live client never drops to zero assigned managers, by revoke or by model change	<code>revoke_manager</code> , <code>apply_model</code>
I6	An <code>mfa_required</code> role contributes nothing at <code>aal1</code>	<code>effective_access</code> and library evaluation, both
I7	The permission model changes only through <code>apply_model</code> , in one transaction, with an event	grants, <code>apply_model</code>
I8	Roles and permissions in use cannot be deleted	foreign keys on <code>delete restrict</code>
I9	Every write for a client serialises on one per-client lock, checks authority after the lock, and treats a <code>request_id</code> retry as either the stored result or <code>request_conflict</code> , no-op first calls included	all write functions, <code>request_log</code>
I10	No caller reads or writes tables directly through PostgREST	grant table asserted at the end of each migration
I11	No claim in the token or user metadata grants anything; only table rows do	<code>effective_access</code> reads tables only
I12	Public enrollment happens once per user per client; later sign-ins and retries never re-grant; a revoked self-assignable role returns only through a manager grant	<code>enrollments</code> primary key, <code>join_client</code>
I13	Promotion to a manager role happens only through <code>bootstrap_manager</code> ; neither a manager call nor a model change promotes existing holders	<code>grant_membership</code> , <code>apply_model</code>

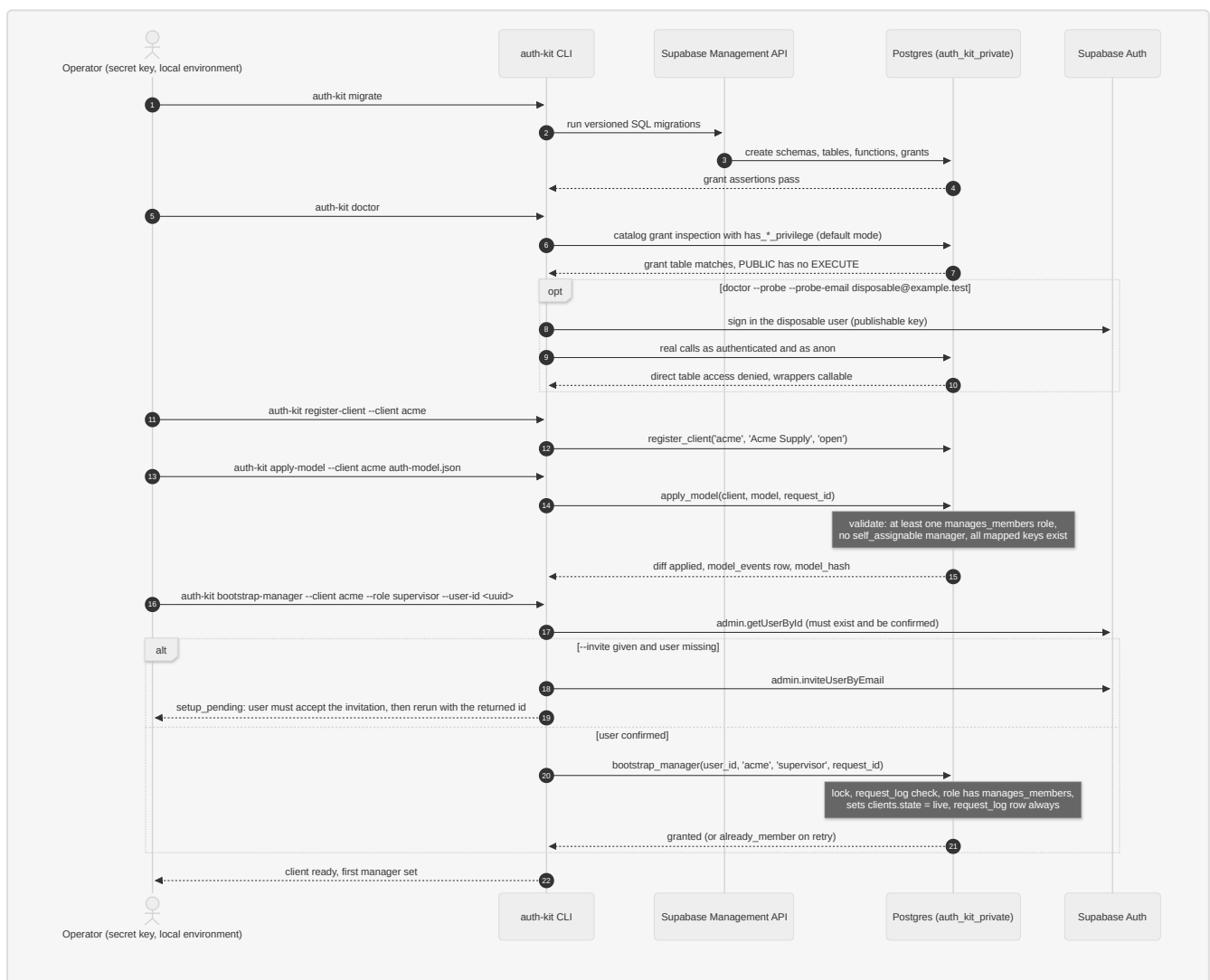
#	Invariant	Where enforced
I14	Every authorization helper, in JS and SQL, evaluates active roles only	packages/core, has_role_impl, has_permission_impl
I15	Manager authority is itself an active-role question: one active manages_members role suffices; a held but MFA-withheld manager role contributes nothing, and mfa_required is returned only when no manager role is active	grant_membership, revoke_membership
I16	auth_kit_private is never API-exposed; its functions are reachable only from SQL, and only with the asserted EXECUTE grants	migration assertion, doctor, hosted test L28
I17	An MFA reset has at most one claim holder at a time; only the holder records the factor list or completes the row, and each request id yields at most one event	mfa_reset_begin, mfa_reset_note, mfa_reset_finish

Not an invariant but a stated limit: on the direct RLS path, Auth session revocation is bounded by access-token expiry (S4b).

6. Sequences

S1. First-time initialisation

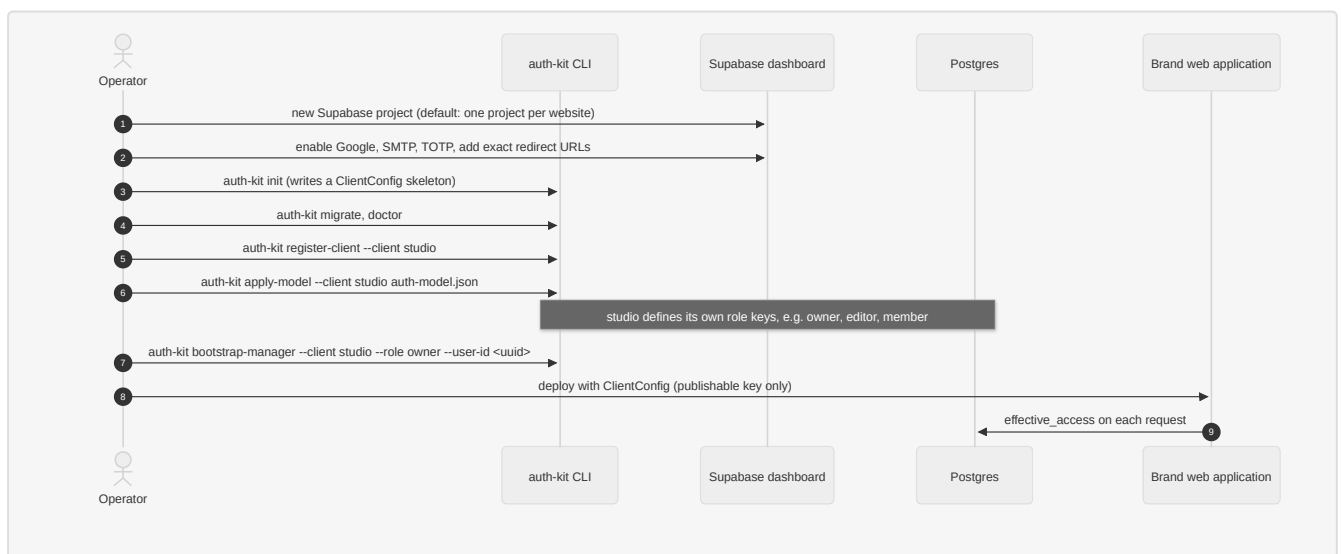
Starting from zero clients and zero roles:



Zero-state edge cases and their exact outcomes:

State	Action	Outcome
Migrated, no client	a user signs up and calls <code>join_client('acme')</code>	<code>unknown_client</code> ; the user is authenticated with no memberships; the browser shows <code>setup_pending</code>
Client registered, no model	<code>join_client</code>	<code>no_default_role</code> ; same pending state; a later retry joins
Client registered, no model	<code>bootstrap_manager</code>	<code>unknown_role</code> ; no row written
Model applied, no manager yet	manager-only actions	nobody can call them; only the operator can act, by design
Two operators run <code>apply_model</code> at once	both	the advisory lock serialises them; the second sees no diff or applies only its remaining delta; two events
<code>bootstrap_manager</code> retried after a lost response	same <code>request_id</code>	one membership, one event, one <code>request_log</code> row; a retry whose first call returned <code>already_member</code> still hits the request row
The operator bootstraps a user whose email is unconfirmed	<code>bootstrap_manager</code>	refused with <code>email_unverified</code>
<code>--email</code> matches zero or several confirmed users	CLI <code>listUsers</code> scan run to the end of the listing	refused <code>unknown_user</code> or <code>ambiguous_user</code> ; the CLI prints how to find the id in the dashboard
<code>--email</code> on a project with more users than the scan cap (10 pages × 1000)	CLI scan reaches the cap with pages remaining	refused <code>lookup_incomplete</code> , nothing written; the message says to use <code>--user-id</code> . One match on page 1 is never accepted while page 11 is unread
Client is <code>live</code> and the model strips <code>manages_members</code> from the only held manager role	<code>apply_model</code>	refused <code>no_manager_would_remain</code> with the holders; nothing written
The model sets <code>manages_members</code> on a role that users already hold	<code>apply_model</code>	refused with the holder list; promotion only via <code>bootstrap_manager</code>
Zero managers before the first bootstrap	any	intentional setup state; <code>state = registered</code> ; the live-client guards do not apply yet

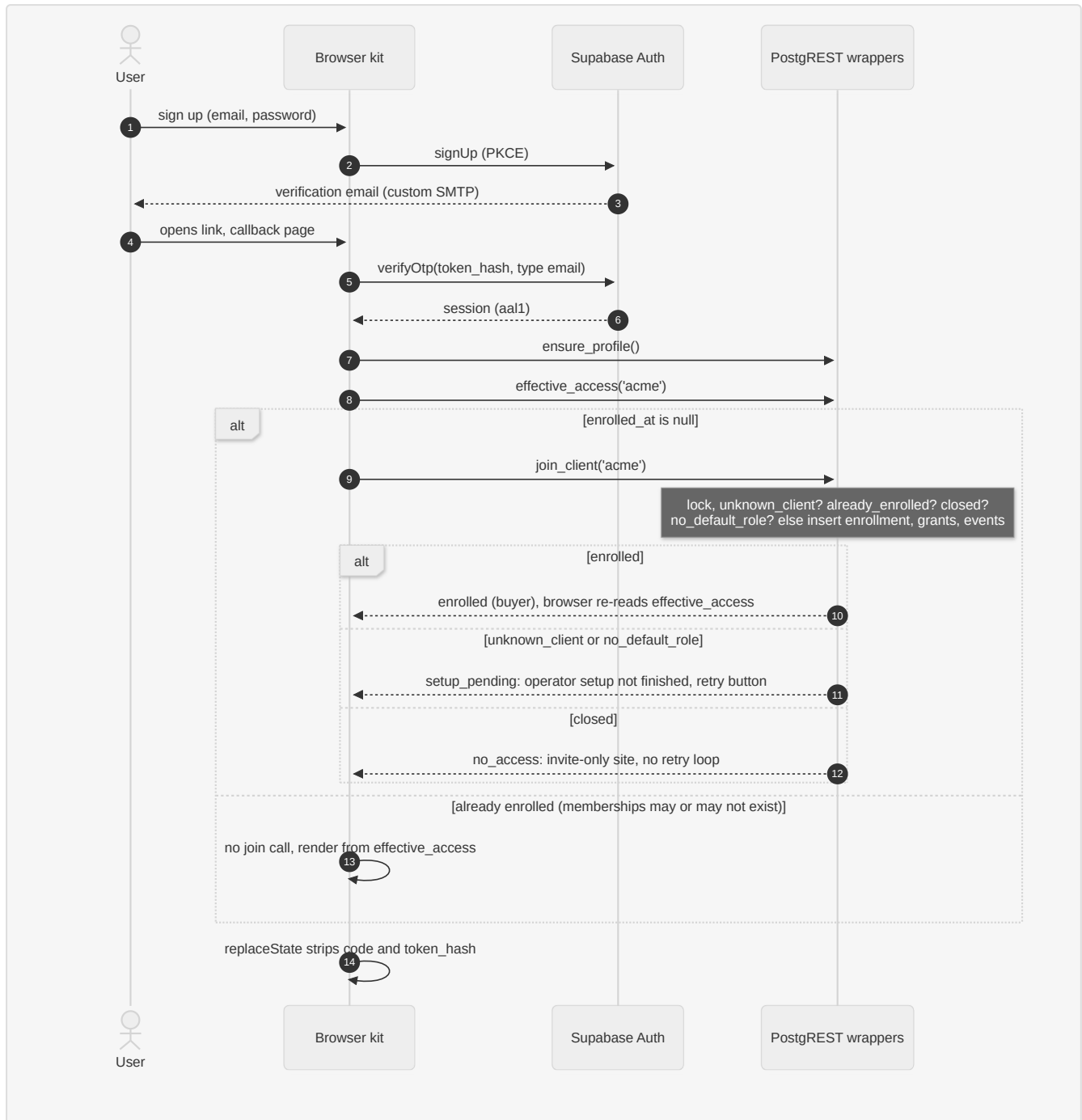
S2. Onboarding a second brand



The alternative, several websites in one project, is the same sequence without the new project and dashboard steps, and with two `register_client` calls. The clients then share one user directory and one provider

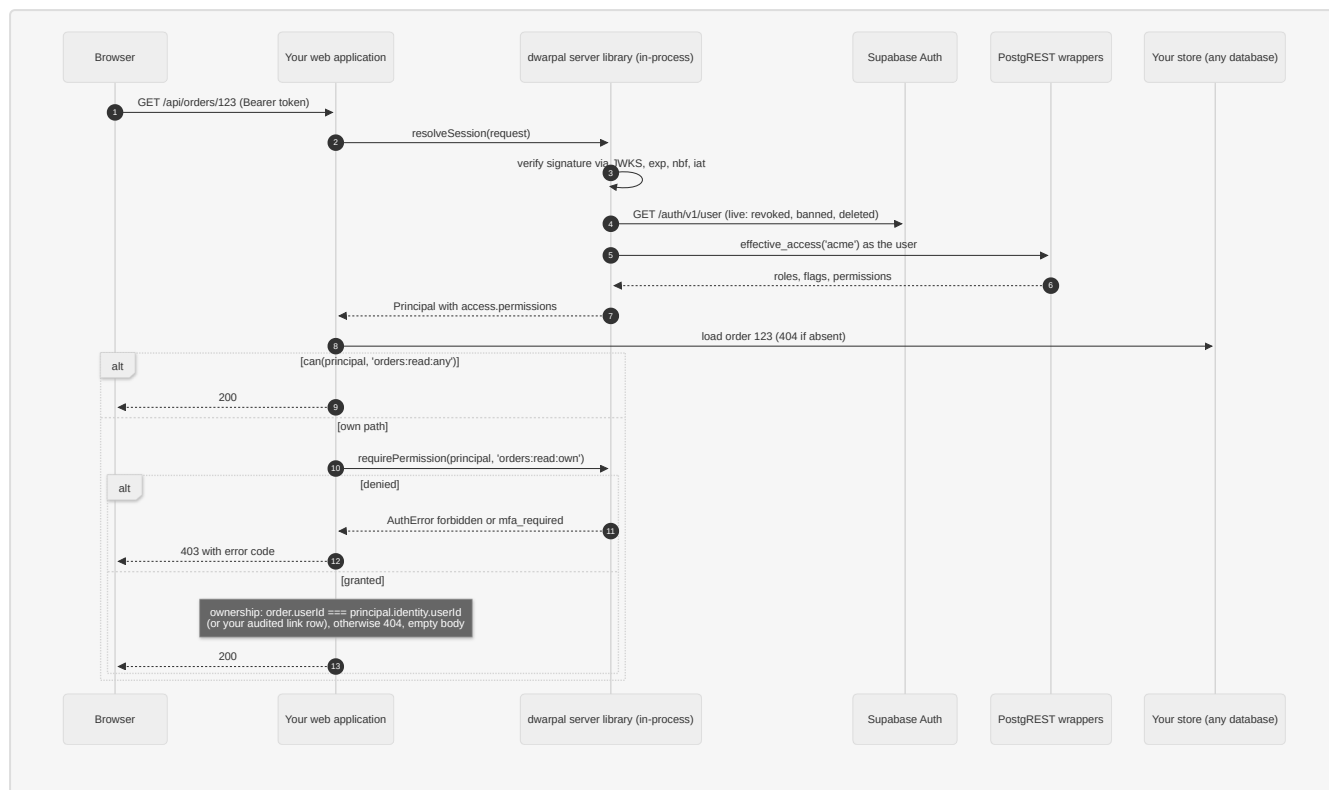
configuration.

S3. Public sign-up and join



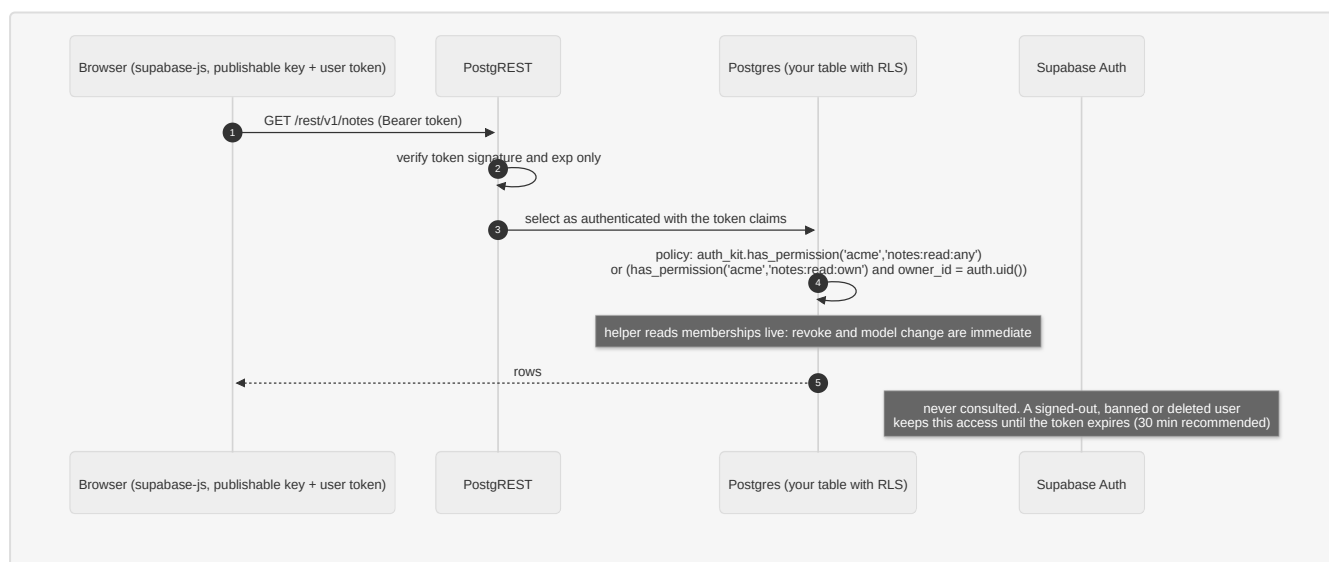
Google sign-in follows the same path from the callback onwards. Join grants only `self_assignable` roles regardless of anything in the sign-up request (I1), and only once (I12): a user whose `buyer` role was revoked by a manager signs in, sees `enrolled_at` set and no membership, and the page shows `no_access`. Nothing the browser does can re-grant it. A user who got `no_default_role` was never enrolled, so a later retry enrolls them once the operator adds a self-assignable role.

S4. Request-time authorization in your web application



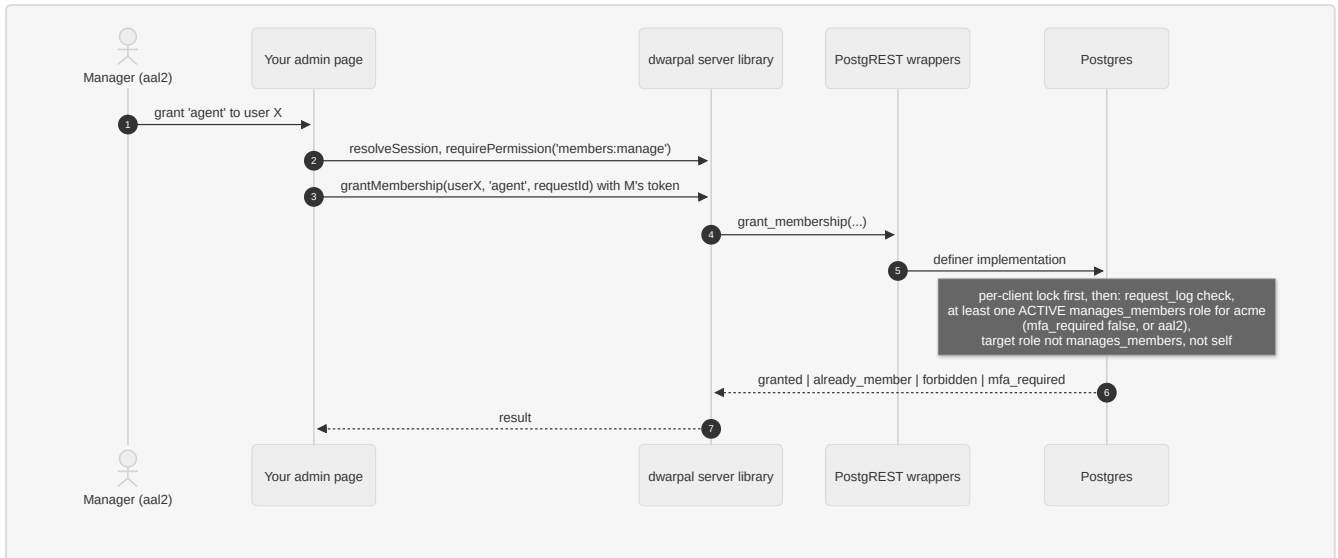
Any failure of the Auth call or the access read yields `unavailable` and a denial. A revoked membership, a changed model, a sign-out, a ban or a deletion takes effect on the next request. **This is the supported path for sensitive records.** The guard checks the broad key first; requiring the narrow key before loading the row would wrongly reject a buyer who holds only `orders:read:own`.

S4b. The direct RLS path, and what it does not check



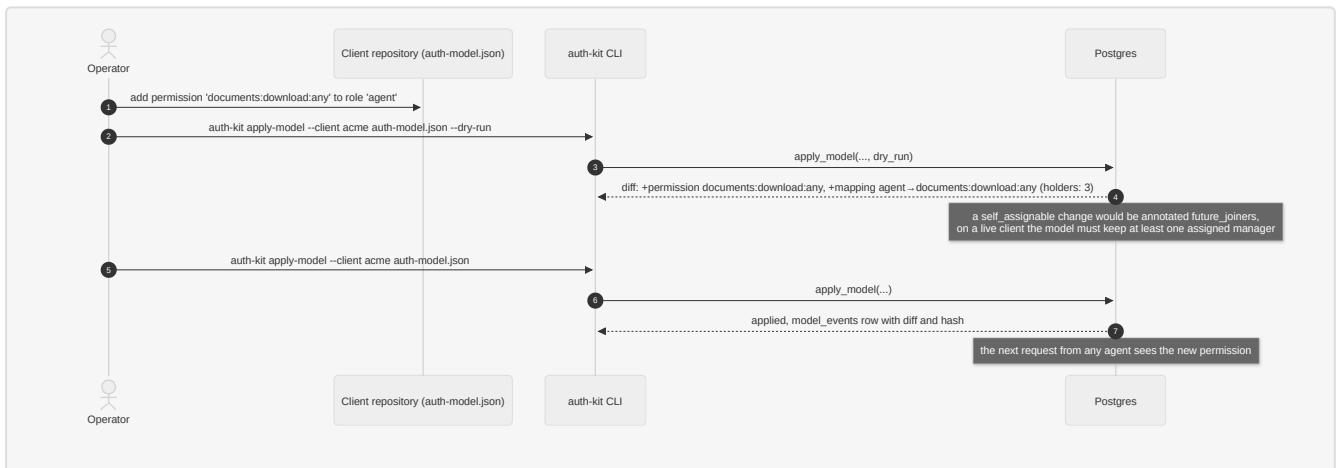
The kit supports this path for public or low-risk data and labels its revocation "bounded by JWT expiry". It never claims one-request revocation for it. The hosted acceptance suite runs the same revoked-session test on S4 and S4b and records both outcomes (test L25).

S5. A manager grants a staff role



The web application never holds the secret key. The manager's own token carries the authority, and SQL rechecks it. A forged actor argument is impossible because the actor is `auth.uid()` inside the function.

S6. Changing the permission model after go-live



Managers manage **people**. Operators manage the **model**. The model is data; the functions that validate and apply it are the contract.

7. The definition path

The permission model for a client is a JSON document, abridged here; [design section 6.1](#) has a full example.

```
{
  "client": "acme",
  "roles": {
    "supervisor": { "manages_members": true, "mfa_required": true,
                    "permissions": ["members:manage", "orders:read:any", "orders:update:any",
                                   "orders:reopen:any",
                                   "documents:review:any", "leads:read:any",
                                   "leads:assign:self"] },
  }
```

```

    "agent":      { "mfa_required": true,
                    "permissions": ["orders:read:any", "orders:update:any",
                                   "documents:review:any",
                                   "leads:read:any", "leads:assign:self"] },
    "buyer":      { "self_assignable": true,
                    "permissions": ["orders:read:own", "documents:upload:own",
                                   "documents:download:own"] }
  },
  "permissions": {
    "members:manage": "grant and revoke non-manager roles",
    "orders:read:own": "read orders the user is linked to (the orders service checks the
link)",
    "orders:read:any": "read any order"
  }
}

```

A permission used by a role but not declared in `permissions` fails validation, so a typo cannot silently create a new permission. (The abridged `permissions` block above would fail for that reason; the full model declares every key.)

How the model can reach the database:

Path	How	Record	Status
A. File and <code>apply-model</code>	the operator runs the CLI at deploy time, like migrations	the file in the client's repository, plus <code>model_events</code>	the default: a reviewable diff, reproducible on the emulator and hosted, rerunnable
B. Programmatic sync from the web application at startup	the same function, called from code	<code>model_events</code> only	not supported: it needs the secret key in the web application's environment
C. Manual calls in the Supabase SQL editor	the operator calls <code>apply_model</code> by hand	<code>model_events</code> only, until reconciled	recovery only. Afterwards run <code>auth-kit export-model > auth-model.json</code> , review and commit before the next ordinary <code>apply</code> , or that <code>apply</code> reverts the change; <code>doctor</code> reports the hash mismatch until then
D. Admin UI	a manager edits the model in a screen	<code>model_events</code>	out of scope: it would let managers change the model and collapse the people/model split

A and C use the same SQL function, so C is never a bypass. The CLI is a convenience over the library's operator operations.

8. Failure modes

Situation	Behaviour
Nth retry of <code>apply_model</code> with the same file	no diff, no new event, exit 0
<code>apply_model</code> fails halfway	nothing written; one transaction
Two operators apply different files concurrently	advisory lock per client; the second applies its delta against the first's result; two events; <code>doctor</code> shows the file hash no longer matches, prompting a rerun
Role deleted from the file while users hold it	<code>apply_model</code> refuses with the list of holders; revoke first

Situation	Behaviour
Permission renamed in the file	treated as remove old plus add new; mappings must move in the same file; one transaction
Model changed while a user has an active session	the next request reads the new model; no cache
Manager revoked mid-action	the SQL recheck inside <code>grant_membership</code> denies; the library's earlier principal never grants anything on its own
Clock skew	affects only token checks (5 s tolerance); the MFA-reset deadline uses the runner's own monotonic clock and never compares clocks across machines
<code>effective_access</code> unavailable	<code>unavailable</code> , <code>denied</code>
User holds an <code>mfa_required</code> role but has not enrolled	permissions from that role withheld; the browser shows <code>mfa_enrol</code> ; other roles work
Secret-key holder writes tables directly	undetectable at write time; <code>doctor</code> reports memberships without events and model-hash drift
Client defines no <code>self_assignable</code> role on purpose	invite-only client; <code>join_client</code> returns <code>no_default_role</code> ; a supported configuration
Manager revokes a self-assignable role; the user signs in again	<code>join_client</code> sees the enrollment row, returns <code>already_enrolled</code> and grants nothing; the page shows <code>no_access</code>
Two first joins for the same user race	both take the per-client lock in turn; the first enrolls, the second returns <code>already_enrolled</code> ; one enrollment row, one event per role
First join fails after inserting the enrollment row	the transaction rolls back; no row; the next join enrolls
The model later adds a new <code>self_assignable</code> role	the dry-run diff says <code>future_joiners</code> ; enrolled users are untouched; managers grant it to existing users if wanted
Same <code>request_id</code> retried with a different payload	<code>request_conflict</code> ; nothing written; the caller must use a new id
First call was a no-op (unchanged model, already a member), then the same id arrives with a different payload	<code>request_conflict</code> , because the no-op wrote a <code>request_log</code> row
<code>mfa-reset</code> dies after deleting one of several factors, before <code>mfa_reset_finish</code>	the <code>pending</code> row keeps <code>factors_seen</code> ; a retry inside the lease gets <code>request_in_progress</code> ; after the lease <code>mfa_reset_begin</code> returns <code>resume</code> with the original list and a fresh <code>run_token</code> ; the remaining factors are deleted (not-found counts as deleted) and finish records both lists; the audit never claims the factors were absent
Two retries of an expired <code>mfa-reset</code> arrive within a second of each other	the lock serialises them; the first claims (new token, <code>started_at = now()</code>) and gets <code>resume</code> ; the second sees a pending row inside the lease and gets <code>request_in_progress</code> before any Auth call; one event
The original <code>mfa-reset</code> runner was only slow and wakes after a takeover completed	its late <code>deleteFactor</code> sees not-found; its <code>note</code> and <code>finish</code> raise <code>run_superseded</code> and write nothing; the completed row and the single event carry the takeover's token
An <code>mfa-reset</code> runner is still going as its lease nears expiry	before each admin API call the CLI checks the claim age against 100 s and stops with <code>lease_expired</code> if exceeded; the row stays <code>pending</code> with its list; the rerun resumes
<code>mfa-reset</code> retried with the same id after success	<code>mfa_reset_begin</code> returns the stored result; no admin API call is made
A completed <code>mfa-reset</code> id is reused for another user	<code>request_conflict</code> from <code>mfa_reset_begin</code> , before any Auth call; the second user's factors are untouched
Two <code>mfa-reset</code> runs with the same id at once	the second waits on the project-wide lock, then sees the <code>pending</code> row inside the lease → <code>request_in_progress</code> before any Auth call

Situation	Behaviour
Manager holds one MFA-free and one MFA-required manager role, at <code>aall</code>	authorised through the active role; the withheld role contributes nothing
Manager holds only an MFA-required manager role, at <code>aall</code>	<code>mfa_required</code> ; nothing written; the page routes to MFA
A consumer or attacker posts to <code>/rest/v1/rpc/<private implementation></code>	PostgREST refuses: the schema is not exposed, for every actor; SQL grants are the second fence for callers already inside SQL
<code>--email</code> scan hits the page cap with more users present	<code>lookup_incomplete</code> ; nothing written; use <code>--user-id</code>
A model change on a live client would strip the last manager, or promote holders	refused before any write; holders listed
A signed-out user reuses an unexpired token on the direct RLS path	rows returned until expiry; a documented limit, not a bug (S4b)
The operator changed the model in the SQL editor and forgot the file	the next <code>apply-model --dry-run</code> shows the revert as a diff; <code>doctor</code> shows hash drift; <code>export-model</code> reconciles

9. Effect on consumers

- The documented check is `requirePermission`; `requireRole` remains as a thin helper that tests active roles only.
- The kit ships no consumer adapters. Each consumer derives its actor from `Principal` (`identity.userId`, `memberships` for the configured client, `access.permissions`, `access.activeRoles`) in its own code and owns its link rows.
- Consumers pin `AUTH_CONTRACT_VERSION === '0.5'` and fail closed on a mismatch.
- There is no member listing in 0.x; a consumer that needs a staff directory fills it from resolved sessions.
- Guards follow the own/any order in section 4.3, and `:self` for actions whose narrow scope is the principal as target.